Introductory Programming with Python 2025
Final Project

Torkil Olsen

December 1, 2025

Aim

The objective of this project is to utilize the topics covered in the course to develop an application based on a set of requirements from a customer. Git version control and adherence to a structured code format, as e.g., outlined in the "Style Guide for Python Code" (PEP8) contribute to the final result. The entire project considers all technologies discussed in the selected sections in chapters 1-9. It is vital to read the entire project description before you begin.

Work Process and Submitting the Project

In the process of solving the assignment, you must use the provided GitHub Classroom repository, so that at appropriate stages the work is committed and pushed to the remote repository — not too seldom, and not unnecessarily often. The code must be well documented — such as by using meaningful variable names, short and straightforward docstrings, as well as concise and precise comments. The purpose of this is to make the code more readable, so attention must also be paid to not over-documenting. It will also be checked whether the code follows the Style Guide for Python Code. Split up the application functionality into appropriate functions and consider splitting functionality into separate files. An example is a "reader.py", which contains all reading functions and is imported into the main program.

The project is to be submitted by committing and pushing to GitHub Classroom repository before the deadline, and by submitting it formally on Moodle.

Application Requirements

The customer who is a store owner needs a simple console application for handling the store's inventory and keeping records of the sales. The customer has data from an old system stored in two .csv files - inventory.csv and sales.csv.

In order to get the application up and running the data from the inventory.csv and sales.csv files needs to be imported. This is a once of operation as in the future the application will only operate with data in .JSON and .txt format. In other words after importing the .csv files they will not be used again. All future operations will operate on the inventory.json and sales.json and they will act as the program's database.

The application will need a menu structure and should continue running until the user chooses to exit.

Program Menu Outline

The program must implement a loop-based text menu with the following options:

1. **Import Data from CSV and Save as JSON**
   - Load `inventory.csv` and `sales.csv`.
   - Load the CSV data into lists/dicts.
   - Save the same data to `inventory.json` and `sales.json`.
   - Handle errors using `try/except`.
2. **Show Inventory**
   - Display all items from the inventory.
   - Show fields: item_id, item_name, category, price, quantity_in_stock.
   - Optional: Show items sorted by name or category
3. **Show Sales**
   - Display all recorded sales line items.
   - Show fields: sale_id, date, item_id, quantity_sold, line_total.
   - Optional: Group by sale_id (to show each order as a block)
4. **Add Item to Inventory**
   - Input: item_id, item_name, category, price, quantity_in_stock.
   - Create a new dictionary with the values.
   - Validate the input and append the new item to the inventory.
   - Update the inventory.json file.
5. **Delete Item from Inventory**

- Ask for the `item_id`.
- If found, remove the item from the inventory.
- Otherwise, display an error message.
- Update the inventory.json file.

6. **Update Item in Inventory**
   - Ask for the `item_id`.
   - If found, display the current item information.
   - Allow the user to modify one or more fields.
   - Update the inventory.json file.
7. **Register a Sale (Basket System)**
   - Start a new sale with a unique `sale_id`.
   - Add items to a basket:
     - Check if the item exists.
     - Check stock before allowing it into the basket.
     - Reduce stock when an item is added.
   - After checkout, write sale line items to the sales list and update JSON files.
8. **Statistics**
   - Display the following computed statistics:
     - **Total number of sales** Count distinct `sale_id` values in the sales list.
     - **Total revenue** Sum all `line_total` values.
     - **Best-selling item** Find the item with the highest total quantity sold.
     - **Average revenue per sale** Compute: total revenue divided by the number of sales.
9. **Backup Inventory**
   - Save the current inventory to inventory_backup.txt in a similar format as shown in Table 1
     - Use tabular format
     - Include the headers for field names
10. **Modify Inventory Backup**
    - Sometimes the quantities get mixed up in the inventory and when that happens it is necessary to update and fix the inventory backup file
      - Read the inventory_backup.txt file
      - Swap the quantity value for two different items in the inventory

- It is `obligatory` to use Regular Expression for matching and replacing the values
- Save the updated inventory to inventory_backup.txt

11. **Exit Program**

- Terminate the application.

Item ID	Item Name	Category	Price	Qty
101	Milk 1L	Dairy	1.25	18
102	Bread Loaf	Bakery	2.50	12
103	Eggs 12-pack	Dairy	3.10	30
104	Apples 1kg	Fruit	2.80	25
105	Bananas 1kg	Fruit	2.20	20
106	Orange Juice 1L	Beverages	3.40	15
107	Coffee Beans 500g	Beverages	6.90	10
108	Tomatoes 1kg	Vegetables	2.60	22
109	Potatoes 2kg	Vegetables	3.20	28
110	Onions 1kg	Vegetables	1.90	19
111	Cheddar Cheese 300g	Dairy	4.50	14
112	Yogurt 4-pack	Dairy	2.95	16
113	Chicken Breast 1kg	Meat	7.80	12
114	Ground Beef 1kg	Meat	8.50	9
115	Pasta 500g	Pantry	1.20	34
116	Rice 1kg	Pantry	2.40	27
117	Cereal 750g	Pantry	3.90	13
118	Chocolate Bar 100g	Snacks	1.10	40
119	Chips 200g	Snacks	2.75	22
120	Bottled Water 1.5L	Beverages	0.95	50
121	Sugar 1kg	Pantry	1.30	21
122	Flour 1kg	Pantry	1.10	18
123	Butter 250g	Dairy	2.60	14
124	Shampoo 500ml	Hygiene	4.20	11
125	Hand Soap 300ml	Hygiene	2.10	17

Table 1: Inventory Overview

Grading

The assignment is split into 3 categories of difficulty for grading purposes.

1. The code should fulfill all the requirements of the project, and menu item 10 (Modify Inventory Backup) is hardcoded to swap the quantity of 2 individual items.
2. The code should fulfill all the requirements of the project, and menu item 10 (Modify Inventory Backup) is dynamically programmed so the user can select items to swap their quantity. When selecting an item, the user should be able to swap the quantity of any other item.
3. The dynamic selection should be available, and an exception should be caught if the user selects an invalid item to swap. When the swap operation is complete, it should be possible to return to the menu and select other processes or a new item to be swapped.

All difficulties have in common that it should be possible to swap the quantity of 2 items with the use of regex.

- To get a grade of up to 7 in the oral exam at least point 1 must be fulfilled, and you must be able to explain thoroughly all parts of the code and what happens when the program is being executed.
- To get a grade of up to 10 in the oral exam at least point 2 must be fulfilled, and you must be able to explain thoroughly all parts of the code and what happens when the program is being executed.
- To get a grade of up to 12 in the oral exam at least point 3 must be fulfilled, and you must be able to explain thoroughly all parts of the code and what happens when the program is being executed.

Furthermore, decent use of version control with Git and a consistent coding style (by following e.g., PEP8) all contribute to the total grading.